

Section D Tasks

Lab Task 1: Employee Record System

Scenario:

An HR department needs an employee database sorted by EmployeeID for fast search, updates, and removal of departed staff.

Task Requirements:

Each node should contain:

- EmployeeID (int, unique key)
- Name (string)
- Department (string)
- Salary (float)

Core Functionalities:

- addEmployee(int id, string name, string dept, float salary)
- findEmployee(int id)
- removeEmployee(int id)
- displayEmployeesInOrder()
- getInorderIDs()

Advanced Operations:

- getMinSalaryEmployee()
- getMaxSalaryEmployee()
- displayEmployeesLevelOrder()

Note: Some testcases have vectors in them. You need to replace them with an appropriate list or array

Google Test Cases:

```
TEST(EmployeeBSTTest, AddAndFindEmployee) {
    EmployeeBST bst;
    bst.addEmployee(10, "Ali", "HR", 60000);
    bst.addEmployee(5, "Sara", "IT", 80000);
    Employee e = bst.findEmployee(5);
    EXPECT_EQ(e.Name, "Sara"); // Expected correct employee details
    EXPECT_EQ(e.Department, "IT");
}

TEST(EmployeeBSTTest, FindNonExistingEmployee) {
    EmployeeBST bst;
    bst.addEmployee(10, "Ali", "HR", 60000);
    Employee e = bst.findEmployee(99);
    EXPECT_EQ(e.Name, ""); // Expected empty when not found
}

TEST(EmployeeBSTTest, DeleteEmployeeLeaf) {
    EmployeeBST bst;
    bst.addEmployee(1, "A", "X", 50000);
    bst.addEmployee(2, "B", "Y", 60000);
    bst.removeEmployee(2);
    EXPECT_EQ(bst.findEmployee(2).Name, ""); // Expected removed
}

TEST(EmployeeBSTTest, DeleteEmployeeWithTwoChildren) {
    EmployeeBST bst;
    bst.addEmployee(50, "Root", "A", 70000);
    bst.addEmployee(30, "L", "B", 40000);
    bst.addEmployee(70, "R", "C", 90000);
    bst.addEmployee(60, "M", "D", 50000);
    bst.removeEmployee(50);
    EXPECT_EQ(bst.findEmployee(50).Name, ""); // Expected root replaced
}

TEST(EmployeeBSTTest, GetMaxSalaryEmployee) {
    EmployeeBST bst;
    bst.addEmployee(1, "A", "HR", 50000);
    bst.addEmployee(2, "B", "IT", 90000);
    Employee e = bst.getMaxSalaryEmployee();
    EXPECT_EQ(e.Name, "B"); // Expected max salary employee
}

TEST(EmployeeBSTTest, InorderSortedIDs) {
    EmployeeBST bst;
    bst.addEmployee(3, "X", "D", 40);
    bst.addEmployee(1, "Y", "E", 50);
    bst.addEmployee(2, "Z", "F", 60);
}
```

```

        vector<int> ino = bst.getInorderIDs();
        EXPECT_EQ(ino, (vector<int>{1,2,3})); // Expected sorted order
    }

    TEST(EmployeeBSTTest, LevelOrderOutput) {
        EmployeeBST bst;
        bst.addEmployee(50, "Root", "A", 70);
        bst.addEmployee(30, "L", "B", 40);
        bst.addEmployee(70, "R", "C", 90);
        vector<int> lvl = bst.getLevelOrderIDs();
        EXPECT_EQ(lvl.front(), 50); // Expected root first in level order
    }

    TEST(EmployeeBSTTest, HeightAndCount) {
        EmployeeBST bst;
        bst.addEmployee(10, "A", "D", 30);
        bst.addEmployee(5, "B", "E", 40);
        bst.addEmployee(15, "C", "F", 50);
        EXPECT_EQ(bst.getHeight(), 2); // Expected height
        EXPECT_EQ(bst.getTotalEmployees(), 3); // Expected count
    }

```

Lab Task 2: Sorted Linked List to Balanced Binary Search Tree Conversion

You are part of a team building a **data indexing module** for a large-scale search engine.

Incoming records are first **sorted and stored as a linked list** (as they arrive sequentially or from an external data source).

However, searching through this linked list is **inefficient ($O(n)$)** for lookups.

To enable **faster searching**, you need to **convert this sorted linked list into a height-balanced Binary Search Tree (BST)**, where:

- In-order traversal of the BST reproduces the original sorted order.
- The tree is as balanced as possible to ensure **$O(\log n)$** search time.

This conversion mimics real-world database and indexing processes — where **sorted data streams are turned into tree-based indices** for efficient query performance.

Data Model

Linked List Node

```
class ListNode {  
  
public:  
  
    int key;  
  
    std::string value;  
  
    ListNode* next;  
  
    ListNode(int k, std::string v) : key(k), value(v), next(nullptr) {}  
  
};
```

BST Node

```
class BSTNode {  
  
public:  
  
    int key;  
  
    std::string value;  
  
    BSTNode* left;  
  
    BSTNode* right;  
  
    BSTNode(int k, std::string v) : key(k), value(v), left(nullptr), right(nullptr) {}  
  
};
```

Requirements:

- Convert the given **sorted linked list** into a **balanced BST**.
- The function should run in **$O(n)$** time — i.e., no repeated traversal of the list.
- Use **in-order construction logic**:

1. Recursively construct the left subtree.
 2. Use the current list node as root.
 3. Move the head pointer forward.
 4. Recursively construct the right subtree.
- Return a pointer to the **root node** of the constructed BST.

You are expected to implement the basic functions for both Linked Lists and BSTs (insertion, deletion, display, etc) and for BSTs you also need to implement the getHeight() and isBalanced() functions.

Test Cases

```
#include "gtest/gtest.h"

#include "SortedListToBST.h"

#include <sstream>

#include <cmath>

#include <string>

TEST(SortedListToBSTTest, BasicLinkedListInsertion) {

    SortedLinkedList list;

    int keys[] = {10, 20, 30, 40, 50};

    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        list.insertSorted(keys[i], "v");
```

```
std::ostringstream out;

list.displayList(out);

std::string expected = "10 -> 20 -> 30 -> 40 -> 50 -> NULL";

EXPECT_EQ(out.str(), expected);
}
```

```
TEST(SortedListToBSTTest, EmptyListConversion) {

    SortedLinkedList list;

    ListNode* head = list.getHead();

    BSTNode* root = convertSortedListToBST(&head, list.countNodes());

    EXPECT_EQ(root, nullptr);
}
```

```
TEST(SortedListToBSTTest, SingleNodeConversion) {

    SortedLinkedList list;

    list.insertSorted(25, "root");

    ListNode* head = list.getHead();
```

```

BSTNode* root = convertSortedListToBST(&head, list.countNodes());

ASSERT_NE(root, nullptr);

EXPECT_EQ(root->key, 25);

EXPECT_EQ(root->left, nullptr);

EXPECT_EQ(root->right, nullptr);
}

```

```

TEST(SortedListToBSTTest, ConvertBalancedListToBST) {

    SortedLinkedList list;

    int keys[] = {10, 20, 30, 40, 50, 60, 70};

    int n = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        list.insertSorted(keys[i], "v");

    ListNode* head = list.getHead();

    BSTNode* root = convertSortedListToBST(&head, list.countNodes());

    std::ostringstream inorder;

    inorderTraversal(root, inorder);

    std::string expected = "10 20 30 40 50 60 70 ";

    EXPECT_EQ(inorder.str(), expected);
}

```

```
    EXPECT_TRUE(isBalanced(root));  
}
```

```
TEST(SortedListToBSTTest, OddEvenSizedLists) {  
  
    // Odd size  
  
    SortedLinkedList oddList;  
  
    int keys1[] = {1, 2, 3, 4, 5};  
  
    for (int i = 0; i < 5; i++)  
        oddList.insertSorted(keys1[i], "v");  
  
    ListNode* head1 = oddList.getHead();  
  
    BSTNode* root1 = convertSortedListToBST(&head1, oddList.countNodes());  
  
    EXPECT_TRUE(isBalanced(root1));  
  
    // Even size  
  
    SortedLinkedList evenList;  
  
    int keys2[] = {1, 2, 3, 4, 5, 6};  
  
    for (int i = 0; i < 6; i++)  
        evenList.insertSorted(keys2[i], "v");  
  
    ListNode* head2 = evenList.getHead();  
  
    BSTNode* root2 = convertSortedListToBST(&head2,  
evenList.countNodes());  
  
    EXPECT_TRUE(isBalanced(root2));  
}
```



```
}
```

```
TEST(SortedListToBSTTest, LargeListConversion) {

    SortedLinkedList list;

    for (int i = 1; i <= 100; i++)

        list.insertSorted(i, "val");

    ListNode* head = list.getHead();

    BSTNode* root = convertSortedListToBST(&head, list.countNodes());

    EXPECT_TRUE(isBalanced(root));

    std::ostringstream inorder;

    inorderTraversal(root, inorder);

    // Check first few elements

    std::string s = inorder.str();

    EXPECT_TRUE(s.find("1 2 3 4 5") != std::string::npos);

}
```

```
TEST(SortedListToBSTTest, StructuralVerification) {
```

```

SortedLinkedList list;

int keys[] = {10, 20, 30, 40, 50, 60, 70};

int n = sizeof(keys)/sizeof(keys[0]);

for (int i = 0; i < n; i++)

    list.insertSorted(keys[i], "v");


ListNode* head = list.getHead();

BSTNode* root = convertSortedListToBST(&head, list.countNodes());


int listCount = list.countNodes();

int bstCount = countBSTNodes(root);


EXPECT_EQ(listCount, bstCount);
}

```

```

TEST(SortedListToBSTTest, InorderMatchesLinkedList) {

    SortedLinkedList list;

    int keys[] = {5, 10, 15, 20, 25, 30};

    int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        list.insertSorted(keys[i], "v");

```

```

ListNode* head = list.getHead();

BSTNode* root = convertSortedListToBST(&head, list.countNodes());

std::ostringstream inorder;

inorderTraversal(root, inorder);

std::string expected = "5 10 15 20 25 30 ";

EXPECT_EQ(inorder.str(), expected);
}

```

```

TEST(SortedListToBSTTest, SkewedListBecomesBalancedBST) {

    SortedLinkedList list;

    for (int i = 1; i <= 10; i++)

        list.insertSorted(i, "v");

    ListNode* head = list.getHead();

    BSTNode* root = convertSortedListToBST(&head, list.countNodes());

    int h = getHeight(root);

    EXPECT_LE(h, 5); // Balanced BST height should be about log2(10)

    EXPECT_TRUE(isBalanced(root));
}

```

